Duelo de Magos

Servidor nivel producción en Azure para un juego por turnos

Proyecto 3 · Ing. Jonathan Solís Parajeles · 12 de noviembre de 2026
Repositorio: https://github.com/mauroza/duelo-magos (etiqueta v1.0.0)



Contenido

1. Resumen y guía de uso
2. Cumplimiento de requisitos
3. Arquitectura y decisiones
4. Modelo de amenazas
5. Rendimiento y optimización
6. Operación, despliegue y observabilidad
7. Costos

Anexo · Capturas del cliente de prueba

1. Resumen y guía de uso

El juego

Cada mago empieza con **12 de vida** y **2 de maná**, y tiene una **afinidad secreta** (fuego, agua o tierra) que el servidor sortea con un generador criptográficamente seguro. En cada ronda **todos eligen en secreto**, y la ronda se resuelve al cerrarse (o al vencer el temporizador):

Acción	Costo	Efecto
Atacar a un mago con un elemento	1 maná	2 de daño · +1 si es tu afinidad · +2 si tu elemento vence al que usó el objetivo · +1 si el objetivo canalizaba
Escudo	1 maná	Bloquea todo el daño; no se puede usar dos rondas seguidas
Canalizar	—	+2 maná (máx. 6). Acción por defecto si se agota el tiempo

Agua apaga fuego, fuego calcina tierra, tierra absorbe agua. Gana el último mago en pie; si caen todos a la vez es empate; tras 15 rondas gana quien tenga más vida. Desconectarse abre un periodo de gracia; si no vuelve, el mago abandona (queda eliminado). **Información oculta:** las jugadas pendientes y las afinidades de los demás nunca salen del servidor hasta que corresponde (la ronda se resuelve o la partida termina).

Correr localmente (menos de 10 minutos)

Requisitos: Docker Desktop (en Windows, con WSL 2).

```
git clone https://github.com/mauroza/duelo-magos.git
cd duelo-magos
docker compose up --build
```

Abre http://localhost:8080/play/ en dos ventanas (una normal y una de incógnito), crea dos cuentas y juega. El ambiente local levanta **2 réplicas del servidor detrás de nginx**, PostgreSQL y Redis: cada jugador puede quedar conectado a una réplica distinta y la partida funciona igual.

Para ver bots jugando contra el servidor local (requiere Node 20+ y `npm ci`):

```
npx tsx tools/bot.ts --url http://localhost:8080 --games 5
```

Pruebas

```
        npm ci
        docker compose up -d postgres redis
        npm test
```

52 pruebas: unitarias del dominio (reglas, daño, temporizadores, vista filtrada), de **integración** con dos réplicas reales, de **concurrencia** (capacidad bajo uniones simultáneas, un jugador en una sola partida, emparejamiento sin duplicados, idempotencia, `STALE_ROUND`), de **seguridad** (tokens, tickets, origen, autorización por partida, información oculta, límites, administración), de **resiliencia** (temporizadores, gracia, abandono, apagado ordenado con reconexión a otra réplica, reinicio total) y de **contratos** (AsyncAPI y OpenAPI sincronizados con el código).

Otros comandos: `npm run lint` · `npm run typecheck` · `npm run build`.

Prueba de carga

```
        docker compose up -d postgres redis
        npm run build
        # servidor sin límites de autenticación para poder registrar miles de usuarios de
prueba
        RATE_LIMIT_AUTH_PER_MIN=100000 RATE_LIMIT_HTTP_PER_MIN=100000 JWT_SECRET=desarrollo-
local-no-usar-en-produccion-0123456789 \
          DATABASE_URL=postgres://duelo:duelo-local-dev@localhost:5432/duelo
REDIS_URL=redis://localhost:6379 node dist/main.js &
        npx tsx load/loadtest.ts --url http://localhost:8080 --matches 25,50,100,200,300 --
rounds 6 --pid $!
```

Resultados y análisis: `docs/performance.md`.

Despliegue en Azure

Todo se despliega con **GitHub Actions** y **Bicep**. No hay contraseñas, claves ni cadenas de conexión: GitHub entra a Azure con OIDC, y el servidor usa identidad administrada para Key Vault, PostgreSQL, Redis, el registro de contenedores y Application Insights.

1. Una sola vez, en Azure Cloud Shell: `infra/bootstrap-azure.sh` (crea los grupos de recursos y la identidad federada de GitHub).
2. En GitHub, crea las variables `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID` y `AZURE_LOCATION`, y los ambientes `dev` y `prod` (prod con aprobación manual).
3. Cada push a `main` ejecuta: CI → **dev automático** → **prod con aprobación**. Un ambiente nuevo se levanta desde cero con una sola ejecución del pipeline.

Rollback en menos de 5 minutos: Actions → **Rollback** → elegir ambiente (o `scripts/rollback.sh prod`). Detalles en `docs/operations.md`.

2. Cumplimiento de requisitos

Mapa entre cada requisito del enunciado y dónde se cumple: código, prueba o documento.

El juego

Restricción	Cumplimiento
Por turnos	Turnos simultáneos: todos eligen en secreto y la ronda se resuelve al cerrarse o al vencer el temporizador (`src/domain/rules.ts`).
Multijugador	2 a 4 magos humanos por partida (`DEFAULT_CONFIG`).
Original	Reglas propias: vida, maná, escudo con enfriamiento, elección de objetivo, afinidad secreta y bonificación por contrahechizo. No es una recreación 1 a 1 de otro juego.
Reglas verificables	El dominio decide sin ambigüedad la validez de cada jugada y el fin de la partida (`validateAction`, `resolveRound`, `checkEnd`). 16 pruebas unitarias.
Información oculta	Jugadas pendientes y afinidades secretas: nunca se envían a otro jugador (`playerView`, eventos con destinatario).

Requisitos funcionales

Capacidad	Dónde	Prueba
Usuarios: registro, sesión, refresh rotativo, perfil	`POST /v1/auth/*`, `GET/PATCH /v1/users/me`	`security.test.ts`
Salas públicas o privadas, listar, unirse, salir, «listo»; inicia con 2 o más listos; capacidad bajo concurrencia	`/v1/matches*`	`concurrency.test.ts`, `gameplay.test.ts`
Emparejamiento: un jugador nunca en dos partidas; espera máxima configurable	`Matchmaker` (líder en Redis, extracción atómica) + `active_membership`	`concurrency.test.ts`
Turnos: solo juega quien corresponde, error tipado, temporizador con acción por defecto	`action.submit` (AsyncAPI)	`rules.test.ts`, `resilience.test.ts`
Conexión: heartbeat, reconexión, gracia configurable y abandono	`Gateway`, `GRACE_MS`	`resilience.test.ts`
Fin de partida: resultado inmutable y repetición jugada por jugada	trigger `matches_immutable`, `GET /v1/matches/{id}/replay`	`gameplay.test.ts`

Capacidad	Dónde	Prueba
Historial paginado; cada quien ve solo lo suyo	`GET /v1/history`	`gameplay.test.ts`
Administración (solo admin y auditada)	`/v1/admin/*`, `audit_log`	`security.test.ts`

Reglas del servidor

Regla	Cómo
Autoritativo	El cliente envía intenciones (`{kind:'attack', element, target}`); el servidor valida y calcula todo.
Orden por partida + secuencia	`SELECT … FOR UPDATE` por partida; cada evento lleva `seq`; una jugada sobre una ronda vieja se rechaza con `STALE_ROUND`.
Idempotencia	`clientMoveId` único por jugador y partida; un reenvío devuelve `ack` con `duplicate: true`.
Vista por jugador	`playerView` y filtrado de eventos por destinatario; la repetición solo muestra eventos públicos o propios.
Durabilidad	Estado y bitácora en PostgreSQL dentro de la transacción; las partidas sobreviven a reinicios (prueba de reinicio total).
Aislamiento	Cada transición es una transacción independiente; los errores se capturan por partida.
Limpieza	Salas inactivas expiran (TTL); al terminar se liberan las membresías.
Reloj y azar del servidor	Plazos guardados en la base y barridos con `SKIP LOCKED`; afinidades con `crypto.randomInt`.

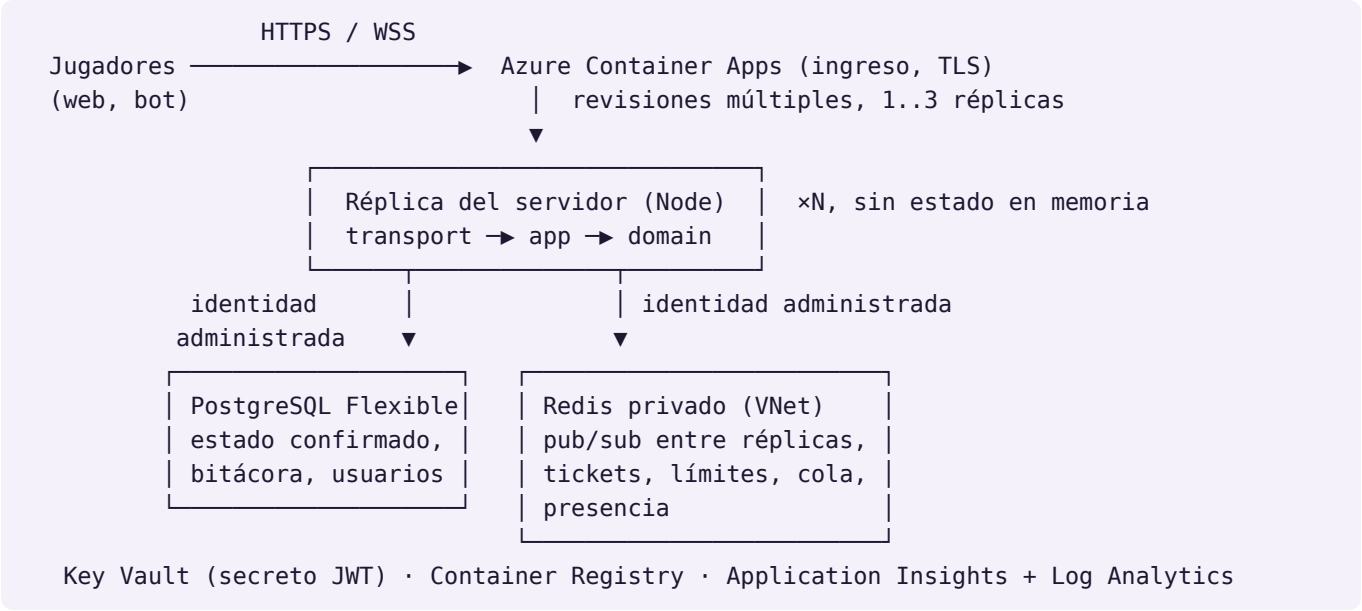
Requisitos técnicos

Requisito	Cumplimiento
Lenguaje tipado estricto	TypeScript con `strict`, `noUncheckedIndexedAccess` y ESLint `strictTypeChecked` (`no-explicit-any: error`); Node 22.
Dominio puro	`src/domain` no importa red, base de datos ni framework (lo impone ESLint).
Tiempo real WSS sin sondeo	WebSocket (`ws`) en `/v1/ws`; el servidor empuja `match.update`.
Protocolo versionado en AsyncAPI 3.0	`contracts/asyncapi.yaml` (validado en CI); sobre `{v:1,…}`.

Requisito	Cumplimiento
API REST en OpenAPI 3.1 en `/docs`	`contracts/openapi.yaml` (validado con Redocly en CI); Swagger UI en `/docs`.
Problem Details (RFC 9457)	`problem.ts`: `application/problem+json`.
Validación de mensajes antes del dominio	Zod estricto; un mensaje inválido recibe `error` sin cerrar la partida.
Heartbeat y cierre con códigos documentados	ping/pong; códigos 4001, 4003, 4008–4012 en AsyncAPI.
Autenticación	JWT firmado con secreto de Key Vault; Argon2id; ticket de un solo uso en el handshake WebSocket.
Autorización	Por endpoint y por mensaje: participante, turno y rol admin.
Límites	429 con `Retry-After`; tasa por conexión WebSocket; tamaño máximo; máximo de conexiones por usuario.
Transporte	Solo HTTPS/WSS; CORS y orígenes WebSocket restringidos; Helmet (CSP, HSTS, `nosniff`).
Secretos	Key Vault con identidad administrada; ninguno en el repositorio (gitleaks en CI).
Cadena de suministro	Dependabot, CodeQL, gitleaks, `npm audit`; imagen con usuario `node`, sin vulnerabilidades críticas (Trivy).
Prueba de carga y metas	p95 de 139 ms con 200 partidas; 0 perdidas y 0 duplicadas; 0 % de error (`docs/performance.md`).
Capacidad y punto de quiebre	Unas 300 partidas por réplica; ~0,3 MB por partida activa.
Mensajes compactos	Eventos delta: 357 B contra 620 B de la vista completa (−42 %).
Optimización medida	De 7 a 4 viajes a la base: p95 −33 %, punto de quiebre +50 %.
IaC	Bicep (`infra/core.bicep`, `infra/app.bicep`); ambiente nuevo con una ejecución del pipeline.
CI/CD	GitHub Actions: CI → dev automático → prod con aprobación manual.
Publicaciones sin cortar partidas y rollback	Revisiones múltiples, drenado con código 4012, `scripts/rollback.sh` y workflow Rollback.
Identidad administrada	ACR, Key Vault, PostgreSQL (solo Entra), App Insights (sin claves locales); OIDC para GitHub.
Observabilidad	OpenTelemetry → Application Insights; logs con `matchId` y `userId`; métricas propias; `/health/live` y `/health/ready`; 3 alertas.
Costo	`docs/cost.md`; dev escala a cero.

3. Arquitectura y decisiones

Vista general



Cualquier réplica puede atender a cualquier jugador de cualquier partida. No hay afinidad de sesión.

Capas

Capa	Carpeta	Depende de	Responsabilidad
Dominio	`src/domain`	nada	Reglas: validar intenciones, resolver rondas, temporizadores, abandono, vista filtrada por jugador. Funciones puras `estado + comando → (estado, eventos)`. ESLint prohíbe importar red, base de datos o framework aquí.
Aplicación	`src/app`	dominio, puertos	Casos de uso: salas, jugadas, presencia, emparejamiento, reloj del servidor, autenticación.
Infraestructura	`src/infra`	—	PostgreSQL, Redis, Key Vault, bus de eventos.
Transporte	`src/transport`	aplicación	HTTP (Fastify) y WebSocket (`ws`). Valida cada mensaje con Zod antes de llegar al dominio.

Flujo de una jugada

1. El cliente envía `action.submit {matchId, clientMoveId, round, action}` por WebSocket.
2. La pasarela aplica el límite de tasa de la conexión, valida JSON, versión y esquema. Un mensaje inválido devuelve `error` sin cerrar la conexión ni tocar la partida.
3. `MatchService.submitAction` abre una transacción y ejecuta `SELECT … FROM matches WHERE id = $1 FOR UPDATE`. Ese bloqueo de fila serializa las jugadas de la misma partida aunque lleguen a réplicas distintas (**orden por partida**). En la misma consulta comprueba si ya existe `(partida, jugador, clientMoveId)` (**idempotencia**).
4. El dominio valida la intención: turno, ronda actual (`STALE_ROUND` si el estado avanzó), maná, objetivo y enfriamiento del escudo. Si todos jugaron, resuelve la ronda.
5. Una sola sentencia con CTE guarda los eventos (bitácora de solo inserción), la jugada, el nuevo estado y la nueva versión. Luego `COMMIT` (**durabilidad**: la jugada confirmada no se pierde).
6. Tras confirmar, publica `{matchId, fromSeq, toSeq, events}` en el canal Redis `match:<id>`.
7. Cada réplica con jugadores de esa partida filtra los eventos por destinatario (**vista por jugador**) y envía `match.update` solo con los cambios (**mensajes compactos**). Si un cliente ve un hueco en las secuencias, pide `match.subscribe` y recibe una vista completa (`match.sync`).

Decisiones de arquitectura (ADR)

ADR-1 · Estado confirmado en PostgreSQL con bloqueo de fila, en lugar de un actor en memoria

Contexto: las jugadas de una partida pueden llegar a réplicas distintas, y una réplica puede reiniciarse en cualquier momento. **Decisión:** la fila de la partida (estado JSONB + versión) es la única fuente de verdad. Cada transición se hace dentro de una transacción con `FOR UPDATE`. **Consecuencias:** orden estricto, durabilidad y supervivencia a reinicios sin coordinación extra. El costo es una transacción por jugada. Se optimizó a 4 viajes a la base (ver `performance.md`) y sostiene unas 300 partidas simultáneas por réplica con p95 < 200 ms.

ADR-2 · Redis para difusión y estado efímero, no para el estado de la partida

Decisión: Redis transporta el pub/sub entre réplicas, los tickets de WebSocket, los límites de tasa, la cola de emparejamiento y la presencia (con caducidad). Perder Redis no pierde jugadas: los clientes se resincronizan desde PostgreSQL.

ADR-3 · Turnos simultáneos con temporizador y acción por defecto

Decisión: todos eligen en secreto. La ronda se resuelve cuando todos los vivos jugaron o cuando vence el plazo; quien no jugó, canaliza. El plazo se guarda en la columna `deadline`. Un barrido en cada réplica usa `FOR UPDATE SKIP LOCKED`, así el reloj sobrevive reinicios y el trabajo se reparte sin duplicarse.

ADR-4 · Eventos de dominio con destinatario

Decisión: cada evento lleva `to: 'all' | <jugador>`. `affinity.assigned` es privado y `action.accepted` solo dice **quién** jugó, nunca **qué**. La pasarela filtra por conexión. La vista completa (`playerView`) también se construye por jugador. Hay pruebas que verifican que ningún mensaje filtra información oculta.

ADR-5 · Autenticación propia con JWT corto y refresh rotativo

Decisión: JWT HS256 de 15 minutos firmados con un secreto de Key Vault, y refresh tokens opacos de un solo uso, guardados como hash. Reutilizar uno ya rotado revoca toda la familia. Contraseñas con Argon2id (parámetros OWASP). El WebSocket se autentica con un ticket de un solo uso de 30 s (`GETDEL` en Redis), para no poner el JWT en la URL.

ADR-6 · Azure Container Apps con revisiones múltiples

Decisión: Container Apps (consumo), con `activeRevisionsMode: Multiple`. Cada despliegue crea una revisión nueva que recibe el tráfico, y luego se desactiva la anterior. Esa revisión cierra sus WebSockets con el código 4012 y los clientes reconectan a la nueva. Como el estado vive en PostgreSQL, las partidas continúan. El rollback es pasar el tráfico a la revisión anterior.

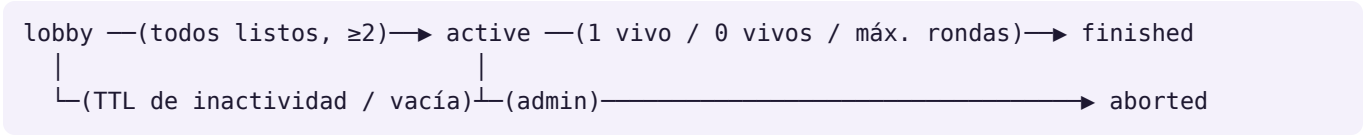
ADR-7 · Redis privado dentro del entorno de Container Apps

Contexto: el plan original era Azure Managed Redis (Balanced B0) con autenticación solo Entra. Al desplegar, Azure respondió `CacheCapacityUnavailable: Balanced_B0 is not currently available in Canada Central due to insufficient capacity`. La suscripción de estudiante solo permite 5 regiones; mover Redis a otra región (Europa o Chile) agregaría entre 100 y 150 ms por operación, y un SKU mayor duplica el costo. **Decisión:** Redis 7 corre como una Container App con **ingreso TCP interno**, dentro de una **VNet** propia del entorno, sin exposición a Internet. La contraseña la genera el pipeline en Key Vault y llega al contenedor como secreto de Container Apps **referenciado a Key Vault y leído con la identidad administrada**: nunca aparece en el repositorio ni en la configuración en texto plano. **Consecuencias:** costo menor (unos 0,25 vCPU) y sin depender de la capacidad regional. Redis solo guarda datos efímeros, así que un reinicio no pierde jugadas: los clientes se resincronizan desde PostgreSQL. El código conserva el modo `REDIS_AUTH=entra` para volver a Azure Managed Redis cuando haya capacidad. Las claves del emparejamiento usan *hash tags* (`{mm}:…`) para funcionar también con Redis en clúster.

ADR-8 · Protocolo JSON versionado

JSON es legible, fácil de depurar y suficiente para el tamaño de los mensajes: 357 B en promedio gracias a los eventos mínimos. Cada mensaje lleva `v: 1`, y una versión desconocida recibe `UNSUPPORTED_VERSION`. MessagePack queda como mejora futura si el ancho de banda se vuelve el cuello de botella.

Ciclo de vida de una partida



Al terminar, se liberan las membresías activas (`active_membership`) y la fila queda **inmutable**: un trigger impide modificar partidas terminadas, y `match_events` y `audit_log` son de solo inserción.

4. Modelo de amenazas

Activos: cuentas y sesiones, integridad de las partidas (resultado justo e inmutable), información oculta (jugadas pendientes y afinidades), disponibilidad del servicio y secretos.

Fronteras de confianza: navegador o bot ↔ ingreso de Azure (TLS) ↔ réplica ↔ PostgreSQL, Redis y Key Vault (red de Azure + identidad administrada) ↔ GitHub Actions (OIDC).

#	Amenaza	STRIDE	Control	Verificación
1	Robo de contraseñas desde la base	I	Argon2id (19 MiB, t=2); se compara contra un hash ficticio para no revelar si el usuario existe	`auth-service.ts`
2	Fuerza bruta de inicio de sesión	S, D	Límite de 10/min por IP en `/v1/auth/*` (429 + `Retry-After`)	`security.test.ts`
3	Robo de un refresh token	S	Rotación en cada uso; reutilizar uno ya usado revoca toda la familia (`TOKEN_REUSED`); solo se guarda el hash SHA-256	`security.test.ts`
4	JWT falsificado o expirado	S	HS256 con secreto de 48 bytes en Key Vault; se validan emisor, audiencia y expiración (15 min)	`security.test.ts`
5	JWT expuesto en URL o logs al abrir el WebSocket	I	Ticket de un solo uso de 30 s (`GETDEL`); los logs censuran `authorization`, `password`, `ticket` y `refreshToken`	`security.test.ts`
6	Secuestro de WebSocket entre sitios (CSWSH)	S	Lista de orígenes permitidos; otro origen se cierra con 4003	`security.test.ts`
7	Jugar en nombre de otro	S, E	El jugador sale del ticket de la conexión, nunca del mensaje	`security.test.ts`
8	Actuar en una partida ajena	E	Cada endpoint y cada mensaje verifican participación (`NOT_A_PARTICIPANT`)	`security.test.ts`

#	Amenaza	STRIDE	Control	Verificación
9	Jugar fuera de turno o con datos falsos (“ahora tengo 10 puntos”)	T	Servidor autoritativo: el cliente envía intenciones y el dominio valida todo contra el estado	`rules.test.ts`
10	Reenvío o duplicado de jugadas (*replay*)	T	`clientMoveId` único por jugador y partida; ronda explícita (`STALE_ROUND`)	`concurrency.test.ts`
11	Ver la jugada o afinidad de otro (trampa por información oculta)	I	Eventos con destinatario; `action.accepted` no revela la acción; la vista se filtra por jugador; la repetición solo muestra eventos propios o públicos	`security.test.ts`, `rules.test.ts`
12	Predecir el azar	T	Afinidades con `crypto.randomInt` (CSPRNG) en el servidor	`match-service.ts`
13	Manipular el reloj	T	Temporizadores en el servidor y guardados en la base; el cliente solo muestra la cuenta regresiva	`resilience.test.ts`
14	Mensajes enormes o malformados	D	Máximo de 4 KiB por mensaje WS (se cierra con 1009) y 16 KiB por cuerpo HTTP; validación Zod estricta (`.strict()`)	`security.test.ts`
15	Inundación de mensajes	D	Token bucket de 10 msg/s con ráfaga de 20 por conexión; 120 req/min HTTP por usuario	`security.test.ts`
16	Agotar conexiones	D	Máximo de 3 WebSockets por usuario (4009); heartbeat que cierra conexiones muertas (4011)	`security.test.ts`

#	Amenaza	STRIDE	Control	Verificación
17	Una partida rota tumba el proceso	D	Cada transición es una transacción aislada; errores por partida capturados; el barrido continúa	`resilience.test.ts`
18	Abuso de administración	E, R	Rol `admin` en el JWT y endpoints con `requireAdmin`; cada acción se guarda en `audit_log` (solo inserción, protegida con trigger)	`security.test.ts`
19	Alterar resultados o historial	T, R	Trigger: una partida terminada es inmutable; `match_events` es de solo inserción	migración `001_init.sql`
20	Secretos filtrados en el repositorio	I	Ningún secreto en el repo: gitleaks en CI; Key Vault con RBAC; producción rechaza `JWT_SECRET` por variable	CI, `config.ts`
21	Credenciales entre servicios	S, I	Identidad administrada para PostgreSQL (solo Entra), ACR, Key Vault y App Insights (`DisableLocalAuth`). Redis: solo red interna (VNet) y contraseña aleatoria de Key Vault entregada con identidad administrada	`infra/*.bicep`
22	Credenciales del pipeline	S, E	OIDC federado (sin secretos en GitHub); la identidad de despliegue solo es Owner de los grupos `dev` y `prod`	`bootstrap-azure.sh`
23	Dependencias o imagen vulnerables	T	`npm audit` (alta y crítica), CodeQL, Dependabot y Trivy (crítica); la imagen corre como usuario `node`, no root	CI

#	Amenaza	STRIDE	Control	Verificación
24	Ataques del navegador (XSS, *clickjacking*, *sniffing*)	T, I	Helmet: CSP `default-src 'self'`, HSTS, `nosniff`, `frame-ancestors`; el cliente escapa todo texto de usuarios	`security.test.ts`
25	Tráfico sin cifrar	I	Solo HTTPS y WSS (`allowInsecure: false`); TLS 1.2+ hacia Redis y PostgreSQL	`app.bicep`

Riesgos aceptados

- **PostgreSQL con acceso público restringido a servicios de Azure** (regla `0.0.0.0`), en lugar de una red privada (VNet). Así se mantiene el costo dentro del crédito de estudiante. La autenticación es solo con Entra ID y no existen contraseñas, por lo que una conexión externa no puede autenticarse. Mitigación futura: integración con VNet y endpoint privado.
- **Valores de desarrollo en el repositorio** (`docker-compose.yml`, pruebas): son credenciales ficticias, sin acceso a nada real. Producción las rechaza y están documentadas en `.gitleaks.toml`.

5. Rendimiento y optimización

Metodología

- **Script:** `load/loadtest.ts`. Es reproducible: los comandos están en el README.
- **Escenario:** N partidas simultáneas de 2 jugadores (2N WebSockets). Cada una juega 6 rondas sin pausa, con decisiones del bot (`chooseAction`).
- **Métrica principal:** por ronda, el tiempo desde que se envía la jugada que **cierra** la ronda hasta que **todos** los jugadores de la partida reciben `round.resolved`. Es la latencia de punta a punta que pide la meta (jugada → todos tienen el nuevo estado).
- **Integridad:** se cuentan las rondas que algún jugador nunca recibió (**perdidas**) y las que recibió más de una vez (**duplicadas**).
- **Recursos:** CPU (% de un núcleo) y RSS máximo del proceso del servidor, leídos de `/proc/<pid>`.
- **Entorno:** 1 réplica (`node dist/main.js`) con PostgreSQL 16 y Redis 7 en Docker, en una máquina de 4 vCPU. El generador de carga corre en la **misma** máquina y compite por CPU, así que los números son conservadores. Los resultados crudos están en `load/results/`.

Metas: p95 < 200 ms · 0 jugadas perdidas o duplicadas · error < 1 %.

Resultado final (versión optimizada, 1 réplica)

Partidas	Conexiones	Rondas medidas	p50	**p95**	p99	Error	Perdidas	Duplicadas	B/mensaje	CPU	RSS
25	50	141	20 ms	**36 ms**	46 ms	0 %	0	0	351	88 %	149 MB
50	100	258	32 ms	**55 ms**	60 ms	0 %	0	0	355	82 %	156 MB
100	200	511	70 ms	**88 ms**	93 ms	0 %	0	0	356	65 %	161 MB
200	400	1029	113 ms	**139 ms**	149 ms	0 %	0	0	357	66 %	201 MB
300	600	1527	165 ms	**204 ms**	211 ms	0 %	0	0	357	59 %	235 MB

✅ Se cumplen las metas hasta **~290 partidas simultáneas por réplica** (580 conexiones).

Punto de quiebre: unas **300 partidas por réplica**, donde el p95 alcanza 204 ms. Más allá la latencia crece linealmente, pero el sistema **no pierde ni duplica jugadas y no produce errores**: se degrada en latencia, no en correctitud.

Costo por partida activa: unos **0,3 MB de RSS** (de 149 MB a 235 MB al pasar de 25 a 300 partidas) y unos **0,25 ms de CPU por jugada** en el servidor. El perfil de CPU (`node --cpu-prof`) mostró que el

tiempo restante se va sobre todo en E/S de sockets (`writev`) hacia PostgreSQL, Redis y los clientes.

Escalado: como el estado vive en PostgreSQL y la difusión pasa por Redis, la capacidad crece agregando réplicas. Container Apps escala de 1 a 3 réplicas en prod. En Azure cada réplica tiene 0,5 vCPU, así que su punto de quiebre esperado es menor (estimado entre 120 y 150 partidas). Con 3 réplicas, entre 360 y 450 partidas.

Optimización 1 · Menos viajes a la base por jugada (latencia)

Hipótesis: el perfil mostraba mucho tiempo en `writev` y una latencia que crecía en forma lineal con la carga. Cada jugada hacía **7 viajes** a PostgreSQL: `BEGIN`, `SELECT … FOR UPDATE`, `SELECT` de idempotencia, `INSERT` de eventos, `UPDATE` de la partida, `INSERT` de la jugada y `COMMIT`.

Cambio (commit `perf: jugadas con 4 viajes…`): - La comprobación de idempotencia viaja como subconsulta dentro del mismo `SELECT … FOR UPDATE`. - Eventos, jugada, nuevo estado y liberación de membresías se escriben en **una sola sentencia** con CTE de escritura. - Total: **4 viajes** (`BEGIN`, `SELECT`, `WITH … UPDATE`, `COMMIT`). - Un caso límite (reenvíos simultáneos de la misma jugada) se resolvió con una segunda consulta que solo se ejecuta en ese caso. La prueba de idempotencia lo cubre.

Partidas	p95 antes	p95 después	Cambio
25	38 ms	36 ms	−5 %
50	55 ms	55 ms	=
100	82 ms	88 ms	ruido
200	209 ms ❌	**139 ms** ✅	**−33 %**
300	— (ya había quebrado)	204 ms	—

Punto de quiebre: de ~200 a ~300 partidas por réplica (+50 %). Archivos: `load/results/delta-*.json` (antes) y `load/results/optimizado-*.json` (después).

Optimización 2 · Mensajes compactos (ancho de banda)

Línea base (`BROADCAST_MODE=full`): tras cada cambio se envía la vista completa de la partida a cada jugador. **Optimizado (`delta`, por defecto):** solo se envían los eventos nuevos, filtrados por jugador. El cliente aplica los cambios y se resincroniza solo si detecta un hueco de secuencia.

Modo	Bytes por mensaje	p95 con 100 partidas	p95 con 200 partidas
full (antes)	620 B	98 ms	162 ms
delta (después)	**357 B (−42 %)**	82 ms	209 ms*

* Medido antes de la optimización 1. La latencia no cambia de forma significativa: el cuello de botella no era la red, sino la base de datos. Por eso la optimización 1 fue la que movió el punto de quiebre. El beneficio de `delta` es el ancho de banda: **42 % menos bytes** por jugador y por cambio, lo que

importa en conexiones móviles. Además, `full` incluye el estado completo en cada mensaje de pub/sub entre réplicas.

Cómo reproducir

```
        # antes de la optimización 1
        git checkout 036d3a3 && npm ci && npm run build
        # después
        git checkout main && npm run build
        # en ambos casos (ver README para variables), opcionalmente con BROADCAST_MODE=full:
        node dist/main.js & npx tsx load/loadtest.ts --matches 25,50,100,200,300 --rounds 6 -
-pid $!
```

6. Operación, despliegue y observabilidad

Pipeline

```
push a main ──► CI ──► dev (automático) ──► prod (aprobación manual en GitHub)
                │        │                    │
                │        └ core.bicep → secreto JWT → imagen → app.bicep → prueba de humo → drenado
                └ lint · tipos · unitarias · integración (Postgres y Redis reales) · AsyncAPI · OpenAPI
                  · npm audit · gitleaks · imagen no root · Trivy · Bicep · CodeQL (workflow aparte)
```

- **Autenticación en Azure:** OIDC (`azure/login`) con una identidad administrada federada. No hay secretos en GitHub; solo variables con identificadores.
- **Ambiente desde cero:** una ejecución del pipeline crea Log Analytics, App Insights, la identidad, ACR, Key Vault, PostgreSQL, Redis, el entorno de Container Apps, el secreto JWT, la imagen, la app y las alertas. El único requisito previo es el grupo de recursos (`infra/bootstrap-azure.sh`).
- **Prueba de humo:** tras desplegar, `/health/ready` y una partida completa entre dos bots (`tools/bot.ts`) contra la URL pública.

Publicaciones sin cortar partidas

1. `app.bicep` crea una **revisión nueva**. Container Apps está en modo `Multiple`, y la revisión nueva recibe el 100 % del tráfico nuevo.
2. La revisión anterior sigue viva con sus WebSockets abiertos. El pipeline la **desactiva**.
3. Al recibir `SIGTERM`, la réplica vieja:
   - deja de estar lista (`/health/ready` responde 503),
   - cierra cada WebSocket con **4012 «server restarting, please reconnect»**,
   - espera hasta 15 s de drenado y libera conexiones.
4. Los clientes reconectan de inmediato (el web reintenta en 300 ms al recibir 4012). Llegan a la revisión nueva, se suscriben de nuevo y reciben `match.sync`.
5. Las partidas continúan: el estado y los temporizadores están en PostgreSQL. El periodo de gracia (30 s) cubre la reconexión, y ninguna jugada confirmada se pierde. La prueba `resilience.test.ts › si una réplica se apaga…` reproduce este escenario.

Rollback (menos de 5 minutos)

Opción A — GitHub: Actions → **Rollback** → *Run workflow* → ambiente (y revisión opcional).

Opción B — terminal (Cloud Shell):

```
./scripts/rollback.sh prod            # vuelve a la revisión anterior
./scripts/rollback.sh prod <revisión> # o a una concreta
```

El script reactiva la revisión destino, espera a que esté `Running`, le pasa el 100 % del tráfico y desactiva la actual con el mismo cierre ordenado. Imprime el tiempo total; lo habitual es 1 a 2 minutos.

Para listar revisiones:

```
az containerapp revision list -n ca-duelo-prod -g rg-duelo-magos-prod -o table --all
```

Observabilidad

- **Trazas, métricas y logs:** OpenTelemetry (`@azure/monitor-opentelemetry`) hacia Application Insights, autenticado con identidad administrada (`DisableLocalAuth: true`). Se instrumentan automáticamente HTTP, PostgreSQL y Redis.
- **Logs estructurados (JSON, pino)** con `traceId`, `spanId`, `instance`, `matchId` y `userId`. Nunca incluyen contraseñas, tokens, tickets ni el encabezado `Authorization` (se censuran).
- **Métricas propias:**

Métrica	Tipo	Descripción
`game.matches.active`	gauge	Partidas en curso (todo el sistema)
`game.ws.connections`	up/down	WebSockets abiertos por réplica
`game.actions`	contador	Jugadas procesadas (`result=accepted|duplicate`)
`game.action.latency`	histograma (ms)	Llegada de la jugada → confirmación, y → difusión (`stage=broadcast`)
`game.rounds.resolved`	contador	Rondas resueltas
`game.matches.finished`	contador	Partidas terminadas, por motivo
`game.rate_limited`	contador	Rechazos por límite de tasa (`transport=http|ws`)

- **Sondas:** `/health/live` (el proceso responde) y `/health/ready` (PostgreSQL y Redis accesibles, y la réplica no está drenando). Container Apps usa ambas, más una sonda de arranque.

Alertas (`infra/app.bicep`)

Alerta	Condición	Severidad
Latencia por jugada	promedio de `game.action.latency` > 200 ms en 5 min	2
Errores HTTP	5xx > 1 % de las solicitudes en 5 min (con al menos 20)	1
Reinicios	más de 3 eventos `BackOff` / `ContainerTerminated` en 15 min	2

Para recibir correos, pasa el parámetro `alertEmail` en `app.bicep`.

Tareas de administración

Para promover un administrador desde Cloud Shell o un contenedor con identidad:

```
npx tsx tools/promote-admin.ts <usuario>
```

Endpoints de administración (todos auditados): - `GET /v1/admin/matches` - `GET /v1/admin/connections` - `POST /v1/admin/matches/{id}/terminate` - `POST /v1/admin/users/{id}/suspend` - `POST /v1/admin/users/{id}/reinstate` - `GET /v1/admin/audit`

7. Costos

Precios de lista aproximados en USD, que pueden variar; se verifican con la calculadora de Azure. El crédito de Azure for Students es de **USD 100**.

Recurso	SKU	prod (24/7)	dev
Container Apps	Consumo, 0,5 vCPU / 1 GiB; prod mín. 1 réplica, dev escala a 0	~$10–15*	~$0–2
PostgreSQL Flexible	Burstable B1ms, 32 GB	~$15	~$15 (o $0 si se detiene)
Redis (Container App interna, 0,25 vCPU / 0,5 GiB)	Consumo, 1 réplica	~$5–8	~$5–8
Container Registry	Basic	$5	$5
Key Vault	Standard, pocas operaciones	< $1	< $1
Log Analytics + App Insights	límite diario de 1 GB (prod) y 0,3 GB (dev)	~$2–5	~$1
Total aproximado		**~$38–49 / mes**	**~$26–31 / mes**

* La concesión gratuita mensual de Container Apps (180 000 vCPU·s y 360 000 GiB·s) cubre gran parte de una réplica de 0,5 vCPU encendida todo el mes.

Cómo mantenerse dentro del crédito

- **dev escala a cero** (`minReplicas: 0`) cuando no hay tráfico.
- **Apagar dev cuando no se usa:**

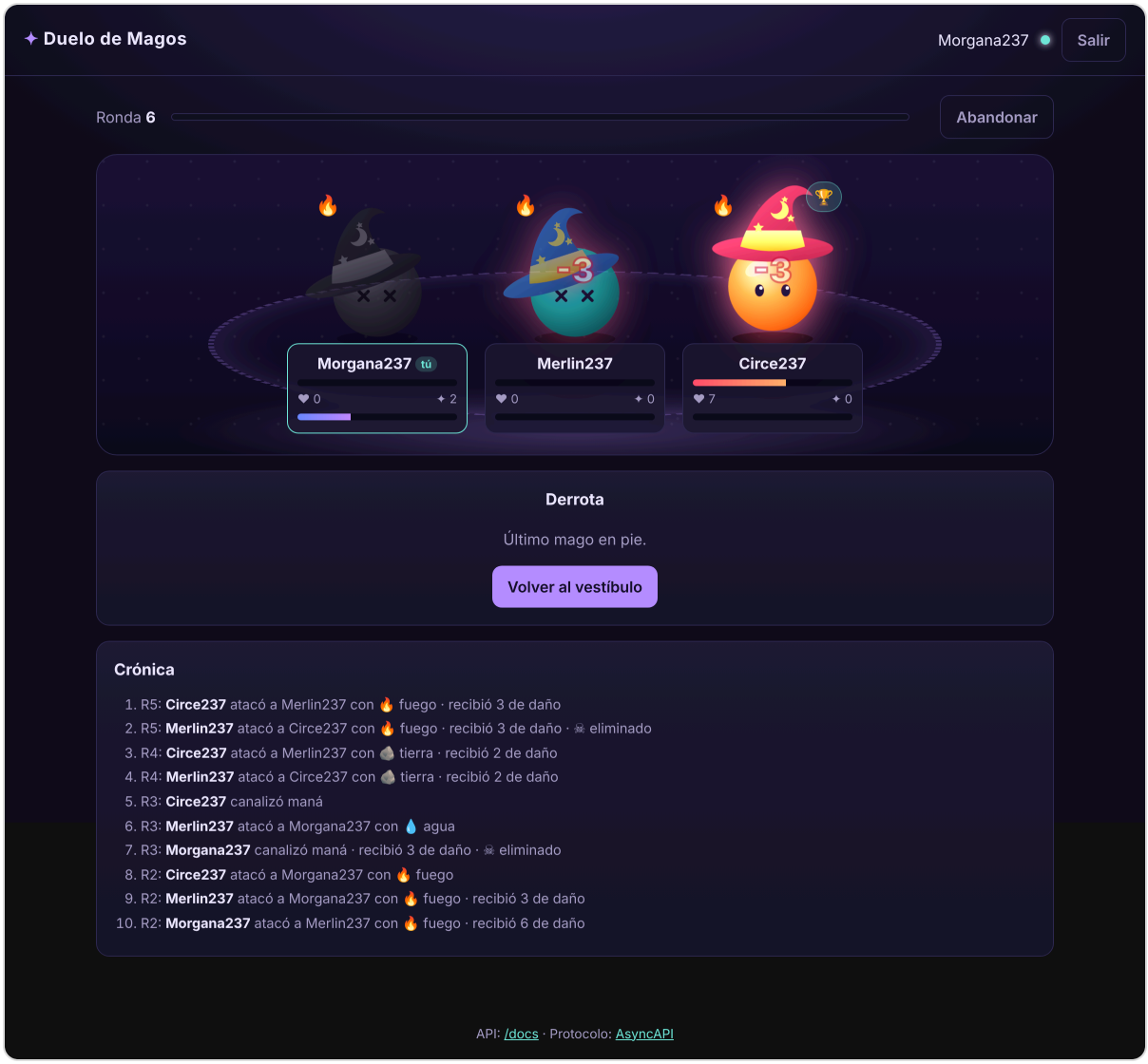
  ```
  az postgres flexible-server stop -g rg-duelo-magos-dev -n <servidor>
  ```

  PostgreSQL se puede detener hasta 7 días seguidos. Redis es una Container App: se puede detener con `az containerapp update -n ca-redis-dev -g rg-duelo-magos-dev --min-replicas 0`.
- **Fuera del periodo de revisión**, eliminar el grupo `rg-duelo-magos-dev` deja el costo de prod en ~$50/mes. El pipeline vuelve a crear dev desde cero cuando haga falta.
- Los límites diarios de ingesta de Log Analytics evitan sorpresas por exceso de logs.
- **Alerta de presupuesto recomendada:** en el portal, Cost Management → Budgets → USD 40/mes con aviso al 80 %.

Anexo · Capturas del cliente de prueba



Hechizos en vuelo



Fin de partida